Applied Software Development Lab
Lab Handout 9: More Control!

Learning objectives
- **Connecting hardware (push button & potentiometer) to a game via the serial interface**
- **Understanding and reusing existing microcontroller code**
- **Establishing the connection between game and controller**
- **How to write a "proper" module**

Looking back and today's goal

So far, you have worked on the Breakout game, which was controlled with the keyboard. Today we extend it – with a self-built **game controller on the breadboard**!

In **lab handouts 4 and 5** you programmed a microcontroller that
- detects the **push-button state** (pressed / released)
- measures the **analog value** of the potentiometer (a **10-bit value**, i.e., an integer in the range **from 0 to 1023**)

This information is sent via the serial interface – for example:
- **Push button pressed**: a line with `A`
- **Push button released**: a line with `a`
- **Voltage change at the potentiometer**: a line such as `X 528`

You can now build this data directly into the game:
- The **push button** fires rockets
- The **potentiometer** controls the **position of the paddle**

Preparation

As usual, we work with two terminals:
- **left**: editing source code
- **right**: compiling code, flashing, opening the serial monitor, testing the game

Cloning the repository

In the left or right terminal:

```
git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm
```

Then, in both terminals, change into the directory with the microcontroller code:

```
cd asp-ulm/controller
```

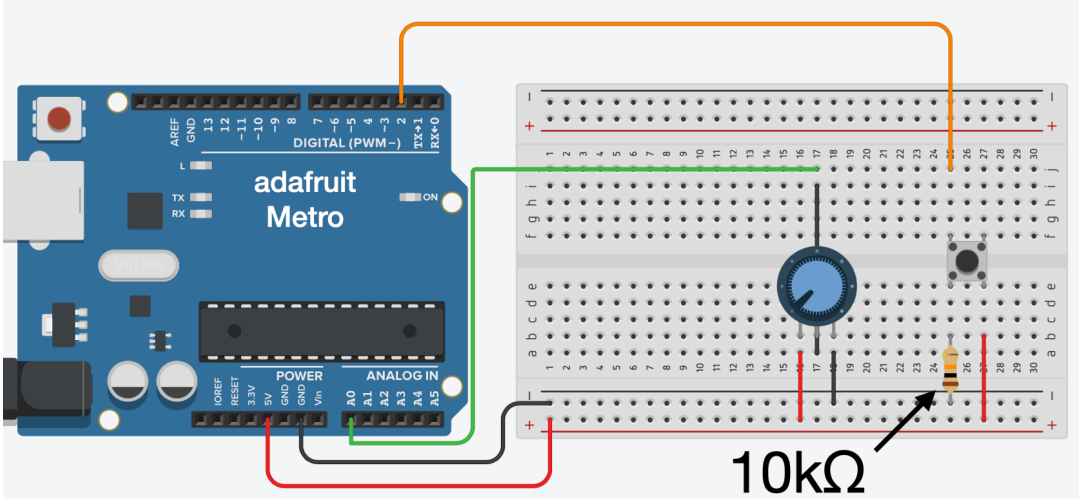
In the next step, we take a closer look at the existing code once more:
- How does the Makefile work?
- How is the code compiled and transferred to the microcontroller?
- How do you open the serial monitor to see the data being sent?

Step 1: Microcontroller code and breadboard tinkering

In the **left terminal**, open the file `controller.ino` containing the C++ code for the microcontroller. Go through the code once more at your own pace and try to understand exactly what happens. If anything is unclear, you can of course
- take another look at lab handout 4,
- ask us,
- or help each other out.

By the way, we like to ask silly questions ourselves now and then – that is part of it 😉

Before you run the microcontroller code, you should rebuild **the familiar circuit** with **push button and potentiometer on the breadboard** and connect it correctly to the microcontroller. Here is a picture of the complete setup (the push button is connected to the digital input **pin 2**, the potentiometer to the analog input **A0**):



The current directory `controller` also contains a file named `Makefile`. If you like, you can briefly display its (somewhat cryptic) content in the **right terminal** with `cat Makefile`. For today, however, it is enough to know **what this Makefile is good for**. It contains rules for automatically invoking the right tools – namely for:
- the **translation** of the C++ code (`controller.ino`) into **machine code** for the microcontroller
- flashing the microcontroller with avrdude
- opening a serial monitor to see the output of the controller

The program `make` does all of this for you – depending on the target you invoke:

Command	Meaning
`make`	Translates the code into machine code (hex file)
`make upload`	Flashes the microcontroller (compiles beforehand if necessary)
`make monitor`	Opens the serial monitor, which shows what the microcontroller sends

⚠️ Important: For `make upload` or `make monitor` to work, **the microcontroller must be connected** and **the USB port must be made visible under WSL** – as usual with: `usb_check`

Task: Once everything is set up and the microcontroller is connected, compile the program, flash the microcontroller, and use the serial monitor to check whether everything works as expected.

© Michael Lehn | CC BY-SA 4.0

Step 2: Reading the serial interface with Python (`serial-reader.py`)

By the way, we also wrote our own serial monitor in Python (see lab handout 4). Let's take another look at it now – this time with fresh eyes!

Preparation
In both terminals, change into the neighboring directory `python`:

```
cd ../python
```

In the **left terminal**, open the file `serial-reader.py`.
In the **right terminal**, you can start the monitor with:

```
python serial-reader.py
```

Task

When we worked on **lab handout 4**, we were not yet very familiar with object-oriented programming. That has changed now – and that is exactly why a second look is worthwhile!

Go through the code `serial-reader.py` step by step – and work on the following tasks:

1. **Object-oriented interface to the serial connection**

   `ser = serial.Serial("/dev/ttyUSB0", 9600)`
   Here, an object of the class `Serial` from the module `serial` is created. This object (which we call `ser`) manages:
   - the serial interface,
   - an (internal) buffer for received bytes,
   - and provides methods for working with it.

   Important methods:
   - `in_waiting`: indicates **how many bytes are currently in the buffer**
   - `read(n)`: reads **n bytes from the buffer** (FIFO – first in, first out)

   In our code, we always read **a single byte** with `ser.read(1)`.

2. **Small code change**

   In the code you will find this line:
   `ch = bytes.decode(ch)`
   Change it to:
   `ch = ch.decode()`
   Then start the program again and check whether everything still works as before. Discuss among yourselves, or ask us, why the new line is equivalent to the old one.

   Note: `ser.read(1)` *returns an object of type* `bytes`*.*

When you are done, make a commit so that the small change to the code is not lost:

```
git commit -a -m "call decode as a method"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 3: Python basics (splitting strings – converting numbers)

Before we build the serial data into the game, we still need a few practical basics in Python.

The problem: The serial interface delivers strings such as `"X 512"`. It would be more practical if we could turn them directly into a list such as `["X", 512]` – that is: split the text at the space and convert the second part into a number.

A short Python session

In the right terminal, start the Python shell with: `python`. Then off you go:

1. Enter:
   `>>> s = "X 512"`
2. Now split the string:
   `>>> l = s.split()`
3. Check the content:
   `>>> s`
   `>>> l`
   → `l` should contain `["X", "512"]`.
   Also try:
   `>>> s = "X    512"`
   `>>> l = s.split()`
   Several spaces make no difference – `split()` ignores superfluous spaces.

4. Access individual list elements:
   `>>> l[0]`
   `>>> l[1]`
5. What happens here?
   `>>> l[1] + 1`
   This produces an error – because `l[1]` is a string, not a number.
6. Convert the string into a number and try again:
   `>>> l[1] = int(l[1])`
   `>>> l[1] + 1`
7. We could also process strings with more than one number – for example:
   `>>> s = "X 523 23"`
   `>>> l = s.split()`
   `>>> for i in range(1, len(l)):`
   `...     l[i] = int(l[i])`
   → Then display `l`.
   You will see: from the second element on, all entries have been converted into integers.

   Why this is useful
   Normally, we only want to program what is necessary – no generalizations in advance. But here we actually do need this generalization – not because of cases with several numbers, but because of cases with no number at all:
   `>>> s = "A"`
   `>>> l = s.split()`
   `>>> for i in range(1, len(l)):`
   `...     l[i] = int(l[i])`
   This works as well – because the loop is not executed at all when `l` has only one element.

© Michael Lehn | CC BY-SA 4.0

Step 4: Our Python interface for the serial interface (`controller.py`)

In the **left terminal**, open the file `controller.py` (it was created in lab handout 5)
and start it in the **right terminal** with: `python controller.py`

Is this object-oriented?
Although a class Controller appears in the code, this is **not** an object-oriented interface – which is quite common for hardware-oriented interfaces.

Instead, there are **two free functions**, which can be used like this:

```
connect("/dev/ttyUSB0")
line = readline()
```

- `connect(port)` must be called first to establish the connection.
- `readline()` returns either:
  - `None` (if no complete line is available yet), or
  - the **received string** (if a line was terminated with `\n`).

What does the class actually do?
The class `Controller` contains two static attributes:
- `Controller.ser` – the serial connection
- `Controller.buffer` – the current input buffer

They serve as **global variables**, but are **clearly encapsulated** by the class – which shows that they belong together logically.
The two functions connect() and readline() access these attributes – so you could also move them directly into the class as static methods and then call them like this:

```
Controller.connect(port)
Controller.readline()
```

You **don't have to** do this – but you **may**, if you feel like trying it out.

Task

1. Now write a function:

   ```
   def readdata():
   ```

   It should try to read a line with `readline()`. If that was successful (i.e., you did not get `None`):

   - split the line into a list as in **Step 3**, i.e., **convert all numbers from the second element on** with `int()`,
   - and return the resulting list.

   If no line is available (`readline()` returned `None`), `readdata()` should also return the value `None`.

   To test it, in the main loop of the program simply replace

   ```
   line = readline()
   ```

   with

   ```
   line = readdata()
   ```

   Then restart the program and check whether everything works as expected – that is: instead of `"A"` or `"a"` you get `['A']` or `['a']`, respectively, and for `"X 523"` you get `['X', 523]`.
2. If everything works:

   ```
   git commit -a -m "serial parser: readdata"
   git push
   ```

© Michael Lehn | CC BY-SA 4.0

Step 5: Turning `controller.py` into a module

So that we can later use our code from other Python programs (e.g., from within the game), we turn controller.py into a proper module.

To do this, we move the previous code from the main loop into a function main() and add the following at the end of the file:

```
if __name__ == "__main__":
    main()
```

What is the benefit?

- If you **run** `controller.py` **directly** with `python controller.py`, the `main()` function is called as before and the program runs. So you can still **test** the module.

- If you **import** the module, e.g., with `import controller`, **nothing is executed automatically**. So you can build the functionality into other places selectively and use it there. That is exactly what we do in the next step!

Task

Quickly test whether everything still works as usual when you run python controller.py. If so:

```
git commit -a -m "turned into module"
git push
```

Step 6: Firing rockets with the push button 🚀

Now we want to test how to build the module `controller.py` into our game – using this example: **The push button fires a rocket.**

1. **Importing the module**
   In the **left terminal**, open the file `breakout.py`. At the top of the code, import the module with

   ```
   import controller
   ```

   If you implemented `connect`, `readline`, and `readdata` as static methods within the class Controller, use: `from controller import Controller`; otherwise you would have to write `controller.Controller.connect` instead of `Controller.connect`.
2. **Establishing the connection to the serial interface**
   Insert the following line before the main loop:

   ```
   controller.connect("/dev/ttyUSB0")
   ```

   If you implemented connect as a static method within the class Controller, use:
   `Controller.connect("/dev/ttyUSB0")`
3. **Processing serial data in the game loop**
   Insert the following code in the main loop:

   ```
   ser_data = controller.readdata()
   if ser_data:
       if ser_data[0] == "A":
           rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
   ```

   Here, too: if you have `readdata()` as a static method in the class, use
   `ser_data = Controller.readdata()`

   The code checks whether a new line has been received from the serial interface. If its first component is `"A"` (push button pressed), a rocket is created and fired.

Test your game. If everything works:

```
git commit -a -m "button fires"
git push
```

© Michael Lehn | CC BY-SA 4.0

Step 7: Controlling the paddle with the potentiometer 🕹️

For now, only one thing is missing: **controlling the paddle directly with the potentiometer.**

For this, we extend the class Paddle with a method `set()` that lets you set the `x` position directly.

1. **Extending the code in `breakout.py`**
   Change the code that queries the controller to:

```
ser_data = controller.readdata()
if ser_data:
   if ser_data[0] == "A":
      rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
   elif ser_data[0] == "X":
       Paddle.set(ser_data[1])
```

2. **Writing the method `set()` in paddle.py**
   Open the file `paddle.py` and add the method:

```
def set(x):
    Paddle.x = x
    if Paddle.x < 0:
        Paddle.x = 0
    if Paddle.x + Paddle.w > Game.width:
        Paddle.x = Game.width - Paddle.w
```

   This sets `Paddle.x` and limits it to the left and right if necessary.
3. **Testing and fine-tuning**
   Start the game and test whether you can move the paddle with the potentiometer.
   If it feels too sluggish, try in breakout.py:

```
Paddle.set(ser_data[1] * 1.5)
```

   or

```
Paddle.set(ser_data[1] * 2)
```

Once everything works (and you are happy with the paddle speed):

```
git commit -a -m "paddle works too"
git push
```

Motivation for the next step: it's about cold hard cash 💰

Now, as a test, unplug the USB cable from the microcontroller and restart the game.

💥 The game crashes because the call `controller.connect("/dev/ttyUSB0")` fails.
Imagine: you have programmed a great game that is already fun without a controller – but only with a controller does it become really cool. And that is exactly what people are willing to spend a few hard-earned euros on.

👉 For this to work, your game must:
- **run smoothly even without a controller**,
- and ideally even allow **connecting the controller while the game is running**.

© Michael Lehn | CC BY-SA 4.0

Step 8: Catching exceptions

We now make our code more robust by cleanly catching errors that occur while establishing the connection.

Task

1. **Adapting `connect()`**
   In controller.py, change the function connect as follows:

```
def connect(port):
    try:
        Controller.ser = serial.Serial(port, 9600)
        Controller.buffer = ""
        print(f"connected to {port}")
    except:
        print(f"could not connected to {port}")
```

   What happens here?
   Here you see the basic pattern:

```
try:
    ...
except:
    ...
```

   This means: the program tries to execute the code in the `try` block. If an error occurs, the program jumps to the `except` block. If no error occurs, the `except` block is simply skipped.

   In our case, this specifically means:
   - If the serial interface can be opened, everything is fine – `Controller.ser` contains a valid object.
   - If it fails, there is just a message, and `Controller.ser` stays `None` – the program does not crash (at this point).

2. **Testing and solving the next problem**
   Start the game **without a connected microcontroller**. You will see: it still crashes – but only later, namely when `readline()` is called. Why? Because this function assumes that `Controller.ser` is always a valid object.

   This is easy to fix. We only need to safeguard `readline()`. At the beginning of the function `readline()`, check whether `Controller.ser` is a valid object:

```
def readline():
    if Controller.ser is None:
        return None
    # rest as before
```

3. **Testing and solving the next problem**
   Please start the game once more, again without connecting the microcontroller. You will notice that it crashes again, but this time at a different place, namely in the function `readdata()`.
   This happens because we previously added a check to `readline()` that tests whether `Controller.ser` is a valid object at all. If it is not, `readline()` returns the value `None`. And that is exactly the problem: in the function `readdata()`, `s.split()` is called immediately without any further check. As long as `s` is a string, this works fine, but if `s` has the value `None`, the program crashes, because a `None` object has no `split()` method.

© Michael Lehn | CC BY-SA 4.0

To prevent this, we need to adapt `readdata()` so that it first checks whether a value is actually present. The function could then look like this, for example:

```
def readdata():
    s = readline()
    if s is not None:
        s = s.split()
        for i in range(1, len(s)):
            s[i] = int(s[i])
        return s
    return None
```

Now the game should also run without a connected microcontroller.

4. **Test: connecting the microcontroller while the game is running**
   As an exception, the **left terminal** is now **not used for editing**, but for running `usb_check` – our familiar workaround that lets WSL see the USB port. We want to test whether the game also works when the microcontroller is connected. We distinguish two cases:

   (a) The microcontroller is already connected before the game starts, or
   (b) it is connected while the game is already running.

   First, test case (a): Connect the microcontroller, run `usb_check` in the left terminal, and then start the game in the right terminal with `python breakout.py`. You should now be able to control the game with the breadboard.

   Next, case (b): Unplug the USB cable from the microcontroller, restart the game in the right terminal with `python breakout.py`, then reconnect the microcontroller and run `usb_check` in the left terminal.
   You will notice: **unfortunately, the breadboard no longer works now**. The reason is simple: our program tries only once, at the beginning, with

   `controller.connect("/dev/ttyUSB0")`

   to open the serial interface. If this fails at startup, it is not tried again later.

   A simple but practical solution is to extend the program so that the user can try to activate the game controller at any time during the game by pressing the **P** key (for “peripheral”). This is not yet real hot plug’n’play, but it is a solution that works in everyday use.

   And this is exactly where you come into play:

   Adapt your program so that this works. When everything works in the end, don’t forget:

```
git commit -a -m "plug and play"
git push
```

© Michael Lehn | CC BY-SA 4.0

Outlook: what awaits you in the next lab handout

The trickiest part is done – congratulations! From now on, it is only about expanding the game further and making it more and more exciting. Here is what awaits you next time:

- There will be bricks that are more robust and have to be hit several times.
- Some bricks have surprising side effects: with some of them you get an extra ball, with others additional spaceships suddenly appear or other exciting things happen.
- And the spaceships? Unfortunately, they are no longer as harmless as before – they start dropping bombs. Luckily, there are also bricks that give you a protective shield.
- And … well, let yourselves be surprised!

The nice thing: You don't have to learn much that is new for this. Most of it will consist of reorganizing the existing code so that everything fits together cleanly. And that is completely normal in programming. At the beginning of a project, you simply cannot foresee every detail – and that is exactly what makes it exciting.

For those who want to experiment on their own

If you feel like implementing your own features before the next lab handout, here are a few ideas on how to proceed. The important thing: always make changes that are as small as possible and that you can test right away. That way, you reach your goal step by step, without everything blowing up in your face at the end.

Global management of balls and spaceships

Currently, the lists `balls` and `ships` are managed locally in `breakout.py`. This means that new balls or spaceships can only be brought into the game there. A good first exercise would be to move these lists into the class `Game` as static attributes. Then you can access them from other modules. This is a small change that can be implemented quickly and tested right away.

Bricks with different effects

In `wall.py` there is the matrix `Wall.brick`. So far, each entry has only the value `0` (no brick) or `1` (normal brick). You could extend the initialization so that the value `2` also occurs there at random. In the function `collision` you can then check what kind of brick was hit: with `1` everything stays as before, with `2` you additionally bring a new ball into the game. Once that works, you will probably notice quickly: you cannot tell by looking at the bricks what is special about them. You can change that by adapting the `draw` function.

Adding spaceships

Similarly, you can initialize bricks with the values `0`, `1`, `2`, or `3`. A brick with the value `3` could, for example, bring a new spaceship into the game.

More robust bricks

You can extend the range of values further, so that a brick with `4` does not disappear after the first collision but is reset to `1`. You then have to hit it twice to destroy it.

Dropping bombs

Bombs can be implemented much like rockets. You could write a class Bomb that is structured analogously to the class Rocket. You could also keep a list of the bombs as a static attribute in the class `Game`, so that it is globally visible.

When you try this out: have fun, and remember to test small intermediate steps regularly! In programming, this is often the decisive trick.

And don't forget to commit your progress to the Git repository. If you create new files, you have to register them once beforehand with `git add FILENAME` – after that, they are part of your project just like the other files.

© Michael Lehn | CC BY-SA 4.0